A01

OOP & JavaFX Shapes

LEGAL NAME	*LEONARD-ADRIAN BUNEA*
PREFERRED NAME	ZOE MCFIFE
TIME SPENT	05:44:33

Table of Contents

Table of Contents 1
Table of Figures 2
Shapes 3
  Vector2 3
  Shape 4
  Polygon 5
    Point 6
    Circle 6
    Rectangle 7
    Line 7
    Triangle 8
    Ellipse 9
    Star 9
JUnit Tests 10
Drawing Shapes 11
  Drawer Class 13
Creative Scene 16

Table of Figures

Figure 1: Vector2 4
Figure 2: Shape Interface 5
Figure 3: Polygon Class 6
Figure 4: Circle Stuff 7
Figure 5: Rectangle Stuff 7
Figure 6: Line Stuff 8
Figure 7: Triangle Constructor 8
Figure 8: Triangle Stuff 8
Figure 9: Ellipse stuff 9
Figure 10: Star Stuff 10
Figure 11: JUnit Tests 10
Figure 12: ShapeApplication 11
Figure 13: Drawable Interface 11
Figure 14: Circle Draw 12
Figure 15: Drawing! 12
Figure 16: Drawer 14
Figure 17: drawShapes but better 15
Figure 18: Picture 15
Figure 19: Creative Scene 16

Shapes

Vector2

To represent positions, I created the Vector2 class that represents, well, a 2D vector. It has a bunch of helper methods to calculate distances between vectors amongst other things. Most of them aren't really needed anyways.

```
public class Vector2
{
    public double x;
    public double y;

    public static final Vector2 ZERO = new Vector2(0, 0);

    public Vector2(double x, double y)
    {
        this.x = x;
        this.y = y;
    }

    public Vector2 add(Vector2 other)
    {
        return new Vector2(this.x + other.x, this.y + other.y);
    }

    public Vector2 subtract(Vector2 other)
    {
        return new Vector2(this.x - other.x, this.y - other.y);
    }

    public Vector2 scale(double scalar)
    {
        return new Vector2(this.x * scalar, this.y * scalar);
    }

    public double dot(Vector2 other)
    {
        return this.x * other.x + this.y * other.y;
    }
```

```
    public double magnitude()
    {
        return Math.sqrt(this.x * this.x + this.y * this.y);
    }

    public Vector2 normalize()
    {
        double mag = magnitude();
        if (mag == 0) return new Vector2(0, 0);
        return new Vector2(this.x / mag, this.y / mag);
    }

    public double distanceTo(Vector2 other)
    {
        return this.subtract(other).magnitude();
    }

    @Override
    public boolean equals(Object other)
    {
        if (this == other) return true;
        if (other == null || getClass() != other.getClass())
return false;
        Vector2 vector2 = (Vector2) other;
        return Double.compare(vector2.x, x) == 0 &&
Double.compare(vector2.y, y) == 0;
    }

    @Override
    public String toString()
    {
        return String.format("Vector2(%.2f, %.2f)", x, y);
    }
}
```

Figure 1: Vector2

I called it Vector2 because I'm used to Godot naming conventions.

Shape

The Shape interface has the area() and perimeter() methods.

```
public interface Shape
{
    double area();
    double perimeter();
}
```

Figure 2: Shape Interface

Polygon

The Polygon class is the base abstract class for all shapes. Each Polygon has a position. Each subclass implements equals() and toString(). For equals() I didn't compare the Polygon positions, because I feel like when comparing, say a Circle, it doesn't matter where the Circle is, if both Circles have the same radius, they're the same. It doesn't really matter this much for this exercise.

```
public abstract class Polygon implements Shape
{
    private Vector2 position;

    Polygon()
    {
       setPosition(Vector2.ZERO);
    }

    Polygon(Vector2 position)
    {
        setPosition(position);
    }

    public Vector2 getPosition()
    {
        return position;
    }

    public void setPosition(Vector2 position)
    {
        this.position = position;
    }

    @Override
```

```
    public String toString()
    {
        return String.format("%s at %s",
getClass().getSimpleName(), getPosition());
    }

    @Override
    public boolean equals(Object obj)
    {
        if (this == obj) return true;
        if (obj == null || getClass() != obj.getClass()) return
false;

        Polygon other = (Polygon) obj;
        return getPosition().equals(other.getPosition());
    }
}
```

Figure 3: Polygon Class

Point

The point is nothing fancy. I didn't reimplement the distanceTo() method here, because the Vector2 class already implements it.

Circle

Circle is also pretty simple.

```
@Override
public double area()
{
    return Math.PI * radius * radius;
}

@Override
public double perimeter()
{
    return 2 * Math.PI * radius;
}

public Circle scale(double scale)
{
    return new Circle(getRadius() * scale);
```

```
}

public boolean isBiggerThan(Circle other)
{
    return this.radius > other.radius;
}
```

Figure 4: Circle Stuff

Rectangle

The rectangle is also simple to implement. Nothing fancy.

```
public double getDiagonalLength()
{
    return Math.sqrt(getWidth() * getWidth() + getHeight() *
getHeight());
}

@Override
public double area()
{
    return getWidth() * getHeight();
}

@Override
public double perimeter()
{
    return 2 * (getWidth() + getHeight());
}
```

Figure 5: Rectangle Stuff

Line

The line accepts a to and from parameter and calculates the midpoint automatically in the constructor when to or from gets changed. The position is set to the midpoint.

```
public Vector2 getMidPoint()
{
    return new Vector2((from.x + to.x) / 2, (from.y + to.y) /
2);
}
```

```
public double getLength()
{
    return Math.sqrt(Math.pow(to.x - from.x, 2) + Math.pow(to.y
- from.y, 2));
}
```

Figure 6: Line Stuff

Triangle

The triangle accepts two sides and an angle in the constructor. It automatically uses ✨ math ✨ to calculate the missing side.

```
Triangle(double a, double b, double angleC)
{
    super();
    setA(a);
    setB(b);

    setAngleC(angleC);

    setC(Math.sqrt(a * a + b * b - 2 * a * b *
Math.cos(Math.toRadians(getAngleC()))));
}
```

Figure 7: Triangle Constructor

```
@Override
public double area()
{
    double s = perimeter() / 2;

    return Math.sqrt(s * (s - a) * (s - b) * (s - c));
}

@Override
public double perimeter()
{
    return a + b + c;
}
```

Figure 8: Triangle Stuff

Ellipse

The ellipse isn't that complicated either.

```
@Override
public double area()
{
    return Math.PI * semiMajorAxis * semiMinorAxis;
}

@Override
public double perimeter()
{
    return Math.PI * (3 * (semiMajorAxis + semiMinorAxis) -
Math.sqrt((3 * semiMajorAxis + semiMinorAxis) * (semiMajorAxis +
3 * semiMinorAxis)));
}
```

Figure 9: Ellipse stuff

Star

The star was a bit more complicated, because I manually calculated how the perimeter formula and I'm a bit rusty with my math. I after just used the formula provided, because I didn't want to double check my own math.

```
public double area()
{
    return getNumPoints() * getInnerRadius() * getOuterRadius()
* Math.sin(Math.PI / getNumPoints());
}

@Override
public double perimeter()
{
    return 2 * getNumPoints() * getEdgeLength();
}

private double getEdgeLength()
{
    return Math.sqrt(getOuterRadius() * getOuterRadius() +
getInnerRadius() * getInnerRadius() - 2 * getOuterRadius() *
```

```
getInnerRadius() * Math.cos(Math.PI / getNumPoints()));
}
```

Figure 10: Star Stuff

JUnit Tests

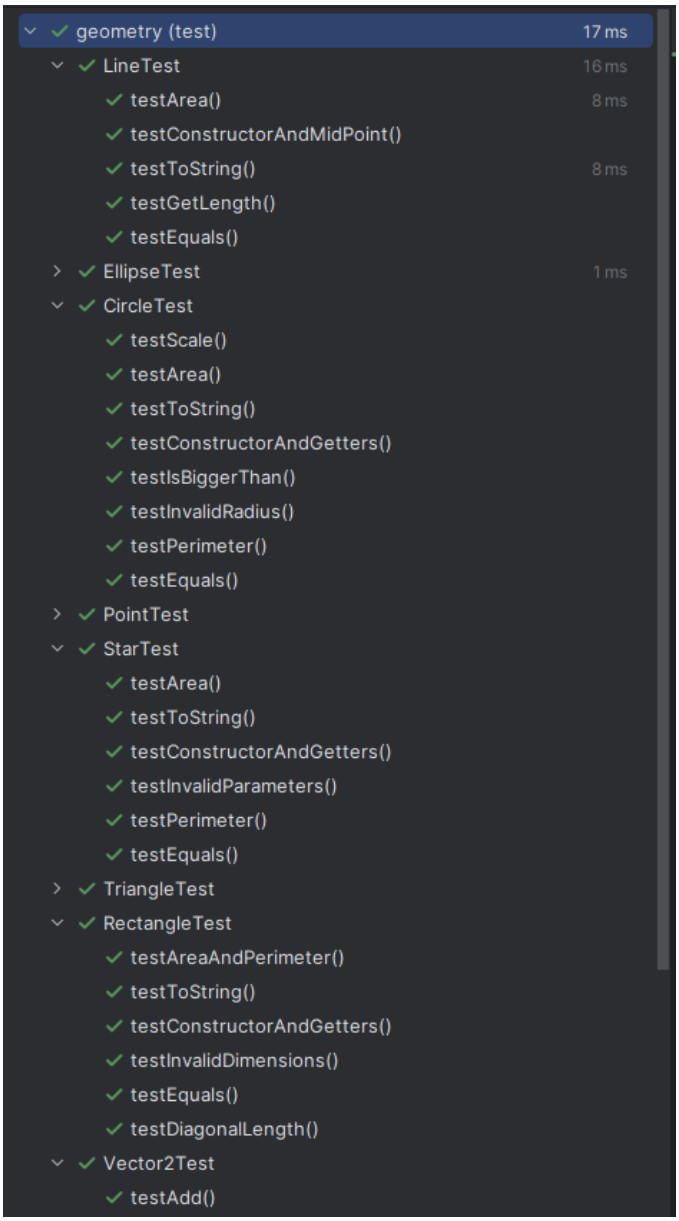

Figure 11: JUnit Tests

I didn't clamp negative values, I instead threw an invalid argument error because I feel like it doesn't make sense to not throw an error there.

Drawing Shapes

First, I migrated my previous project to a new JavaFX project and removed the FXML stuff because I don't need to mess around with that just yet.

```
public class ShapeApplication extends Application
{
    @Override
    public void start(Stage stage) throws IOException
    {
        stage.setTitle("Drawing Operations Test");
        Group root = new Group();
        Canvas canvas = new Canvas(800, 600);

        GraphicsContext gc = canvas.getGraphicsContext2D();
        drawShapes(gc);

        root.getChildren().add(canvas);
        stage.setScene(new Scene(root));
        stage.show();
    }
```

Figure 12: ShapeApplication

I added another interface for the Polygon class.

```
public interface Drawable
{
    void draw(GraphicsContext gc, Color color);
}
```

Figure 13: Drawable Interface

Each shape implements its own draw method.

```
@Override
public void draw(GraphicsContext gc, Color color)
{
    gc.setFill(color);
    gc.fillOval(getPosition().x - radius, getPosition().y -
radius, radius * 2, radius * 2);
}
```

Figure 14: Circle Draw

```
private void drawShapes(GraphicsContext gc)
{
    Point point = new Point(new Vector2(100, 100));
    point.draw(gc, javafx.scene.paint.Color.RED);

    Circle circle = new Circle(new Vector2(400, 300), 150);
    circle.draw(gc, javafx.scene.paint.Color.GREEN);

    Ellipse ellipse = new Ellipse(new Vector2(600, 200), 100,
50);
    ellipse.draw(gc, javafx.scene.paint.Color.BLUE);

    Line line = new Line(new Vector2(200, 500), new Vector2(700,
200));
    line.draw(gc, javafx.scene.paint.Color.ORANGE);

    Rectangle rectangle = new Rectangle(new Vector2(300, 400),
200, 100);
    rectangle.draw(gc, javafx.scene.paint.Color.PURPLE);

    Star star = new Star(new Vector2(500, 100), 25, 50, 5);
    star.draw(gc, javafx.scene.paint.Color.YELLOW);

    Triangle triangle = new Triangle(new Vector2(160, 500), 60,
90, 50);
    triangle.draw(gc, javafx.scene.paint.Color.CYAN);
}
```

Figure 15: Drawing!

Drawer Class

```
public class Drawer
{
    private final List<Polygon> polygons;
    private final GraphicsContext gc;

    public Drawer(GraphicsContext gc)
    {
        polygons = new ArrayList<>();
        this.gc = gc;
    }

    public Drawer(GraphicsContext gc, List<Polygon> polygons)
    {
        this.polygons = polygons;
        this.gc = gc;
    }

    public void drawAll()
    {
        polygons.forEach(polygon -> polygon.draw(gc,
getRandomColor()));
    }

    public void draw(int index)
    {
        draw(index, getRandomColor());
    }

    public void addPolygon(Polygon polygon)
    {
        polygons.add(polygon);
    }

    public void draw(int index, Color color)
    {
        if (index < 0 || index >= polygons.size())
        {
            throw new IndexOutOfBoundsException("Index must be
between 0 and " + (polygons.size() - 1));
        }
```

```
        polygons.get(index).draw(gc, color);
    }

    private Color getRandomColor()
    {
        return Color.color(Math.random(), Math.random(),
Math.random());
    }

    public List<Polygon> getPolygons()
    {
        return polygons;
    }
}
```

Figure 16: Drawer

```
private void drawShapes(GraphicsContext gc)
{
    Point point = new Point(new Vector2(100, 100));
    Circle circle = new Circle(new Vector2(400, 300), 150);
    Ellipse ellipse = new Ellipse(new Vector2(600, 200), 100,
50);
    Line line = new Line(new Vector2(200, 500), new Vector2(700,
200));
    Rectangle rectangle = new Rectangle(new Vector2(300, 400),
200, 100);
    Star star = new Star(new Vector2(500, 100), 25, 50, 5);
    Triangle triangle = new Triangle(new Vector2(160, 500), 60,
90, 50);

    Drawer drawer = new Drawer(gc);

    drawer.addPolygon(point);
    drawer.addPolygon(circle);
    drawer.addPolygon(ellipse);
    drawer.addPolygon(line);
    drawer.addPolygon(rectangle);
    drawer.addPolygon(star);
    drawer.addPolygon(triangle);
```

```
        drawer.drawAll();
    }
```

Figure 17: drawShapes but better

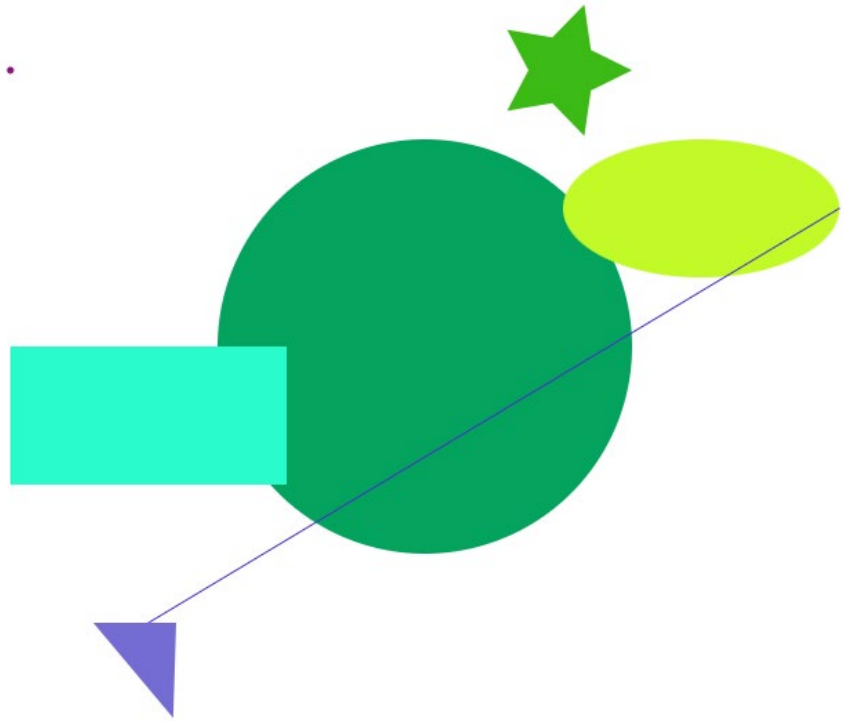

Figure 18: Picture

Creative Scene

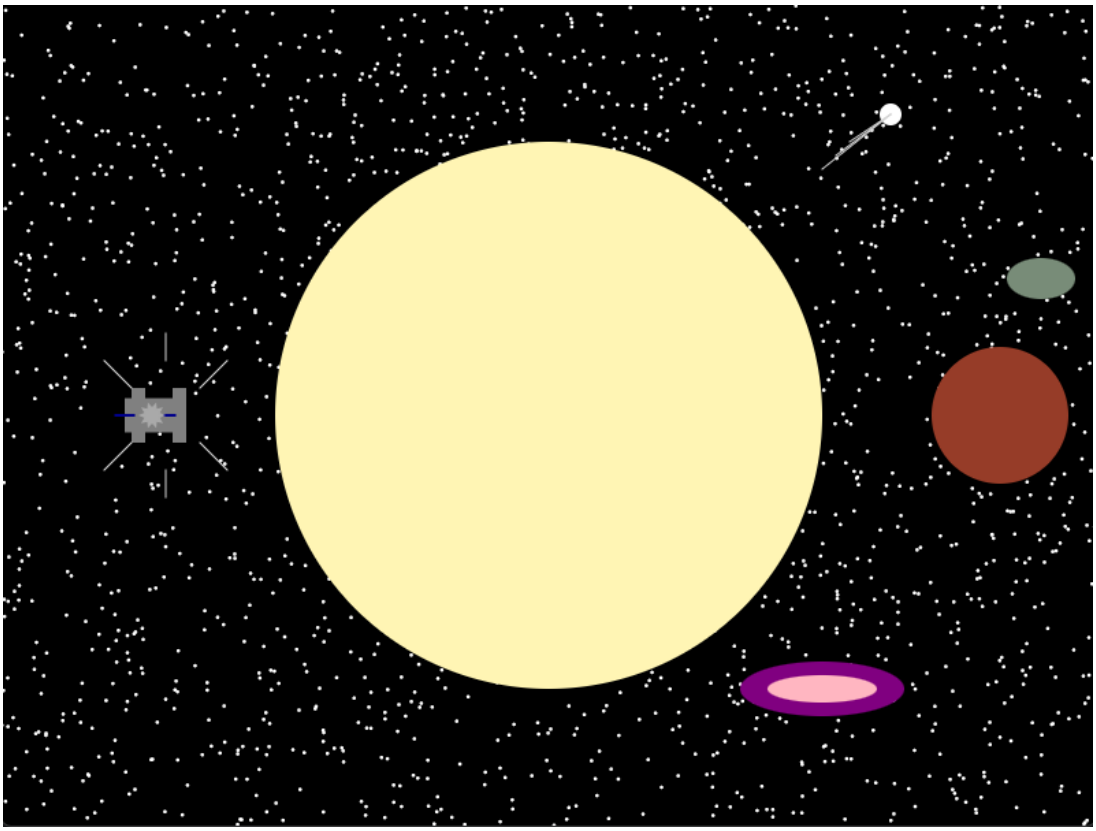

Figure 19: Creative Scene

I decided to make some random space scenery. Background is big black rectangle and the stars are randomly generated points. Star is a circle, planet is a circle, but it has a moon that is egg shaped. Left of the star I made a sorta space station that just a bunch of rectangles and there's a star in the middle. Also a bunch of lines around because why not! There's also a comet and a galaxy? here too.